



472.97

Общий рейтинг

OTUS

Развиваем технологии, обучая их создателей

[Подписаться](#)

badcasedaily1

22 авг 2023 в 11:42

Извлечение признаков из текстовых данных с использованием TF-IDF



10 мин



72K

Блог компании OTUS, Python*, Анализ и проектирование систем*

Изучение текстовых данных является одной из фундаментальных задач в области анализа данных и машинного обучения. Однако тексты представляют собой сложные и многомерные структуры, которые не могут быть напрямую обработаны алгоритмами машинного обучения. В этом контексте извлечение признаков — это процесс преобразования текстовых данных в числовые векторы, которые могут быть использованы для обучения моделей и анализа. Этот шаг играет ключевую роль в предварительной обработке данных перед применением алгоритмов.

Term Frequency-Inverse Document Frequency (TF-IDF) — это один из наиболее распространенных и мощных методов для извлечения признаков из текстовых данных. TF-IDF вычисляет важность каждого слова в документе относительно количества его употреблений в данном документе и во всей коллекции текстов. Этот метод позволяет выделить ключевые слова и понять, какие слова имеют больший вес для определенного документа в контексте всей коллекции.

Основы TF-IDF

Понятие и формула TF-IDF

Термины "TF" (Term Frequency) и "IDF" (Inverse Document Frequency)

TF (Частота термина) обозначает, насколько часто определенное слово появляется в данном документе. Таким образом, TF измеряет важность слова в контексте отдельного документа.

IDF (Обратная частота документа) измеряет, насколько уникально слово является по всей коллекции документов. Слова, которые появляются в большинстве документов, имеют низкое IDF, так как они не вносят большой информационной ценности.

Формула вычисления TF-IDF

Формула TF-IDF комбинирует понятия TF и IDF, чтобы вычислить важность каждого слова в каждом документе. Формально, формула выглядит следующим образом:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) * \text{IDF}(t)$$

РЕКЛАМА



Ты точно знаешь себя?

Проверь свой IT-профиль



- **IDF(t)** - Обратная частота документа (IDF) для слова "t".

Пример расчета TF-IDF

Подготовка корпуса текстов и токенизация

Для наглядности давайте представим, что у нас есть набор документов, составляющих наш корпус текстов. Давайте рассмотрим несколько простых документов:

1. "Машинное обучение — это интересная область."
2. "Обучение с учителем — ключевой аспект машинного обучения."
3. "Область NLP также связана с машинным обучением."



Откройте новый уровень в карьере

удаление стоп-слов, приведение к нижнему регистру и токенизация — разбиение текстов на отдельные слова или токены.

Расчет TF-IDF для слов в корпусе

Представим, что мы хотим вычислить TF-IDF для слова "машинное" в первом документе. Давайте предположим, что TF для этого слова равен 1 (поскольку оно встречается 1 раз в данном документе), а IDF можно вычислить как общее количество документов (3) деленное на количество документов, в которых встречается это слово (2). Таким образом, IDF для слова "машинное" равен $\log(3 / 2) = 0.18$.

Теперь мы можем вычислить TF-IDF для слова "машинное" в первом документе: $TF-IDF = 1 * 0.18 = 0.18$.

Продолжая этот процесс для каждого слова в каждом документе, мы можем создать матрицу TF-IDF, где строки представляют слова, а столбцы - документы.

Преимущества и ограничения TF-IDF

Преимущества использования TF-IDF для извлечения признаков

TF-IDF предоставляет несколько ключевых преимуществ:

1. **Учет важности слов:** TF-IDF учитывает как частоту слова в документе, так и его общую редкость по всей коллекции. Таким образом, он помогает выделять ключевые слова, которые часто встречаются в данном документе, но не слишком распространены в остальных.
2. **Устранение шума:** Слова, которые встречаются в большинстве документов (стоп-слова), имеют низкий IDF и, следовательно, низкий общий вес TF-IDF. Это позволяет устранить шум и сосредоточиться на более важных словах.

Ограничения метода и ситуации, в которых он может быть неэффективен

1. **Отсутствие семантической информации:** TF-IDF не учитывает семантические связи между словами, что может привести к ограниченной способности понимания смысла текста.
2. **Чувствительность к длине документа:** Длинные документы могут иметь более высокие значения TF, даже если ключевые слова встречаются реже. В таких случаях, TF-IDF может недооценить важность конкретных слов.

Важно понимать, в каких ситуациях TF-IDF будет эффективен, а когда стоит рассмотреть



Ты точно знаешь себя?

Проверь свой IT-профиль

Применение TF-IDF в задачах анализа текстов

А. Извлечение ключевых слов и терминов

Выделение наиболее важных слов в тексте

Извлечение ключевых слов из текстовых данных является одним из наиболее распространенных сценариев использования TF-IDF. Одним из способов сделать это - это выбрать топ-N слов с наибольшими значениями TF-IDF. Давайте рассмотрим пример на Python:

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Пример текстовых данных
documents = [
    "Машинное обучение - это интересная область.",
    "Обучение с учителем - ключевой аспект машинного обучения.",
    "Область NLP также связана с машинным обучением."
]

# Создание объекта TfidfVectorizer
tfidf_vectorizer = TfidfVectorizer()

# Применение TF-IDF к текстовым данным
tfidf_matrix = tfidf_vectorizer.fit_transform(documents)

# Получение списка ключевых слов и их значения TF-IDF для первого документа
feature_names = tfidf_vectorizer.get_feature_names_out()
tfidf_scores = tfidf_matrix.toarray()[0]

# Сортировка слов по значениям TF-IDF
sorted_keywords = [word for _, word in sorted(zip(tfidf_scores, feature_names), reverse=True)]

print("Ключевые слова:", sorted_keywords)
```

Объяснить код с  SourceCraft

Применение ключевых слов в поисковых системах

Извлечение ключевых слов с помощью TF-IDF позволяет улучшить поиск и индексацию текстовых данных. Представьте, что вы хотите построить поисковую систему для коллекции документов. Вы можете извлечь ключевые слова для каждого документа с помощью TF-IDF и использовать их для индексации и ранжирования результатов поиска.

В. Кластеризация и категоризация текстовых данных

Группировка текстов по сходству на основе TF-IDF

TF-IDF также может быть использован для кластеризации текстовых данных, то есть группировки схожих документов в один кластер. Кластеризация может помочь выявить общие темы и понять структуру данных. Рассмотрим следующий пример:

```
from sklearn.cluster import KMeans

# Применение кластеризации KMeans к матрице TF-IDF
num_clusters = 2
```



Ты точно знаешь себя?

Проверь свой IT-профиль

```
# Показать примеры документов в каждом кластере
for cluster_id in range(num_clusters):
    cluster_indices = np.where(kmeans.labels_ == cluster_id)[0]
```

```
print(f"Кластер {cluster_id + 1}:")
for idx in cluster_indices:
    print(documents[idx])
print("-----")
```

[Объяснить код с SourceCraft](#)

Практические примеры применения кластеризации

- *Новостные порталы:* Новостные статьи могут быть автоматически категоризированы по темам с использованием кластеризации на основе TF-IDF.
- *Социальные сети:* Кластеризация сообщений или постов пользователей может помочь создать персонализированные ленты новостей.

С. Классификация текстов

Обучение модели классификации на основе TF-IDF

TF-IDF также может быть использован для классификации текстов на основе их содержания. Для этого мы можем обучить модель машинного обучения на векторах TF-IDF, представляющих документы, и затем использовать эту модель для предсказания категории новых текстов. Пример:

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

# Пример данных для классификации
categories = ["технологии", "спорт", "политика"]
labels = [0, 1, 2]
data

= [...] # Список текстовых данных
X_train, X_test, y_train, y_test = train_test_split(data, labels, test_size=0.2, random_state=0)

# Преобразование текстовых данных в матрицу TF-IDF
tfidf_vectorizer = TfidfVectorizer()
X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)
X_test_tfidf = tfidf_vectorizer.transform(X_test)

# Обучение модели классификации
classifier = MultinomialNB()
classifier.fit(X_train_tfidf, y_train)

# Предсказание категорий для тестовых данных
y_pred = classifier.predict(X_test_tfidf)

# Оценка точности модели
accuracy = accuracy_score(y_test, y_pred)
print(f"Точность модели: {accuracy}")
```

[Объяснить код с SourceCraft](#)

Ты точно знаешь себя?

Проверь свой IT-профиль

- *Автоматизация клиентского обслуживания:* Классификация текстовых запросов клиентов в автоматически создаваемые категории может помочь улучшить обслуживание клиентов.

- *Фильтрация спама*: Классификация электронных писем на "спам" и "не спам" на основе содержания текста.

Дополнительные методы и улучшения TF-IDF

А. Нормализация и взвешивание TF-IDF

Нормализация по длине документа

Когда мы вычисляем TF-IDF для слов в документах разной длины, длинные документы могут иметь более высокие значения TF, что может повлиять на точность результатов. Чтобы избежать этой проблемы, можно нормализовать значения TF-IDF по длине документа. Пример на Python:

```
from sklearn.preprocessing import normalize

# Нормализация матрицы TF-IDF по длине документа
tfidf_matrix_normalized = normalize(tfidf_matrix, norm='l2', axis=1)
```

Объяснить код с  SourceCraft

Взвешивание TF-IDF с использованием различных схем

Как итоговое значение веса для каждого слова, TF-IDF может быть взвешено различными схемами, такими как окончательная нормализация, сглаживание и другие методы, которые могут учитывать специфику задачи.

В. Использование N-грамм

Определение N-грамм и их роль в извлечении признаков

N-граммы представляют собой последовательности из N смежных элементов (слов или символов) в тексте. Их роль в извлечении признаков заключается в том, чтобы улавливать семантические отношения между словами, которые идут вместе. Например, в случае биграмм (N=2), "искусственный интеллект" имеет смысл, отличный от отдельных слов "искусственный" и "интеллект".

Применение N-грамм вместе с TF-IDF

Мы можем комбинировать использование N-грамм и TF-IDF для более точного извлечения признаков из текста. Давайте рассмотрим пример:

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Пример текстовых данных
documents = [
    "Искусственный интеллект становится все более распространенным.",
    "Интеллектуальные системы обладают большим потенциалом."
]

# Создание объекта TfidfVectorizer с использованием биграмм
tfidf_vectorizer = TfidfVectorizer(ngram_range=(1, 2))
```



Ты точно знаешь себя?

Проверь свой IT-профиль

```
# Получение списка фичей и их значения TF-IDF для первого документа
```

```
feature_names = tfidf_vectorizer.get_feature_names_out()
tfidf_scores = tfidf_matrix.toarray()[0]

# Сортировка фичей по значениям TF-IDF
sorted_features = [feature for _, feature in sorted(zip(tfidf_scores, feature_names), reverse=True)]

print("Важные фичи:", sorted_features)
```

Объяснить код с  SourceCraft

С. Сравнение с другими методами извлечения признаков

Сопоставление TF-IDF с Word2Vec, Doc2Vec и другими методами

Существует множество других методов извлечения признаков из текста, таких как Word2Vec, Doc2Vec, FastText и многие другие. Эти методы, в отличие от TF-IDF, учитывают семантические связи между словами и векторное представление слов. Поэтому для некоторых задач, особенно связанных с семантическим анализом, они могут быть более эффективными.

Выбор подходящего метода в зависимости от задачи

Выбор метода извлечения признаков зависит от конкретной задачи и характеристик текстовых данных. TF-IDF подходит хорошо для задач, связанных с извлечением ключевых слов, кластеризацией и классификацией. Однако для более сложных задач, где важны семантические отношения между словами, стоит рассмотреть использование более современных методов, таких, как Word2Vec.

Практические советы по использованию TF-IDF

А. Подготовка и предобработка текстовых данных

Удаление стоп-слов и специальных символов

Перед применением TF-IDF к текстовым данным, часто полезно провести предварительную обработку данных. Одним из широко распространенных шагов является удаление стоп-слов - слов, которые не несут смысловой нагрузки (например, предлоги, союзы) и специальных символов. Для этого можно использовать библиотеку Natural Language Toolkit (NLTK) на Python:

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import string

# Загрузка стоп-слов и пунктуации
nltk.download('stopwords')
nltk.download('punkt')
stop_words = set(stopwords.words('russian'))
punctuation = set(string.punctuation)

def preprocess_text(text):
    words = word_tokenize(text.lower()) # Привести к нижнему регистру и токенизировать
    # Удаление стоп-слов и специальных символов
    filtered_words = [word for word in words if word not in stop_words and word not in punctuation]
```



Ты точно знаешь себя?

Проверь свой IT-профиль

```
# пример предобработки текстовых данных
text = "Машинное обучение - это интересная область, изучаемая многими."
```

```
preprocessed_text = preprocess_text(text)
print("Предобработанный текст:", preprocessed_text)
```

Объяснить код с  SourceCraft

Приведение к нижнему регистру и лемматизация

Кроме удаления стоп-слов, также полезно привести текст к нижнему регистру и выполнить лемматизацию - приведение слов к их базовой форме. Это позволяет уменьшить разнообразие форм слова и улучшить качество извлекаемых признаков. Воспользуемся библиотекой spaCy для лемматизации:

```
import spacy

# Загрузка языковой модели spaCy
nlp = spacy.load("ru_core_news_sm")

def preprocess_and_lemmatize(text):
    doc = nlp(text.lower()) # Привести к нижнему регистру и лемматизировать
    lemmatized_words = [token.lemma_ for token in doc if token.text not in punctuation and token.is_alpha]
    return " ".join(lemmatized_words)

# Пример предобработки и лемматизации текста
text = "Машинное обучение - это интересная область, изучаемая многими."
preprocessed_lemmatized_text = preprocess_and_lemmatize(text)
print("Предобработанный и лемматизированный текст:", preprocessed_lemmatized_text)
```

Объяснить код с  SourceCraft

В. Выбор параметров TF-IDF

Выбор веса TF и IDF

В sklearn's TfidfVectorizer, вы можете настроить параметры `sublinear_tf` и `use_idf`, которые влияют на способ вычисления TF и IDF. `sublinear_tf=True` делает вес TF менее линейно растущим, что может быть полезным для учета насыщенности текста ключевыми словами.

Определение порога для отбора ключевых слов

После применения TF-IDF можно выбрать пороговое значение для веса, чтобы определить, какие слова считать ключевыми. Это может быть полезно, чтобы ограничить количество ключевых слов и избежать шума. Пример:

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Пример текстовых данных
documents = [...]

# Создание объекта TfidfVectorizer
tfidf_vectorizer = TfidfVectorizer()
```



Ты точно знаешь себя?

Проверь свой IT-профиль

```
# Определение порога для отбора ключевых слов
threshold = 0.2
```

```
important_words = [word for word, score in zip(tfidf_vectorizer.get_feature_names_out(), tfidf_
print("Важные

слова:", important_words)
```

[Объяснить код с !\[\]\(3d8c13c92b853674f749aac6fa869926_img.jpg\) SourceCraft](#)

С. Работа с большими объемами данных

Эффективное хранение и обработка матрицы TF-IDF

При работе с большими объемами данных матрица TF-IDF может стать очень большой и занимать много памяти. Для оптимизации можно использовать разреженные матрицы, которые хранят только ненулевые значения:

```
from scipy.sparse import csr_matrix

# Пример преобразования в разреженную матрицу
sparse_tfidf_matrix = csr_matrix(tfidf_matrix)
```

[Объяснить код с !\[\]\(e8fb589d58dad1692debababa5e928b6_img.jpg\) SourceCraft](#)

Параллельные вычисления для ускорения процесса

При обработке больших объемов текстовых данных можно использовать параллельные вычисления для ускорения процесса вычисления TF-IDF. Для этого можно использовать параметр `n_jobs` в `TfidfVectorizer`:

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Создание объекта TfidfVectorizer с параллельными вычислениями
tfidf_vectorizer = TfidfVectorizer(n_jobs=-1)

# Применение TF-IDF к текстовым данным
tfidf_matrix = tfidf_vectorizer.fit_transform(documents)
```

[Объяснить код с !\[\]\(4688aadfd656ded00cd6bdfae55089a9_img.jpg\) SourceCraft](#)

Заключение

Использование TF-IDF открывает перед нами широкий спектр возможностей для анализа и интерпретации текстовых данных. Совместно с современными методами и инструментами анализа, он помогает в создании информативных моделей и понимании семантических связей в текстах. При использовании правильных методов предобработки и параметров TF-IDF, вы сможете добиться высокой точности и эффективности в анализе текстовых данных.

Спасибо за чтение статьи. Надеемся, что полученные знания помогут вам с успешным применением в ваших будущих проектах.



Ты точно знаешь себя?

Проверь свой IT-профиль

Если вы хотите глубже понять процессы анализа требований и бизнес-процессов, курс «Специализация Системный и бизнес-анализ» даст вам системное представление о современных методах работы.

А тем, кто настроен на серьезное системное обучение, рекомендуем рассмотреть Подписку — выбираете курсы под свои задачи, экономите на обучении, получаете профессиональный рост. Узнать подробнее

Теги: python, td-idf, аналитика, изменения требований

Хабы: Блог компании OTUS, Python, Анализ и проектирование систем

+11 93 4

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Электронпочта

Подписаться

Оставляя почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок

 OTUS
Развиваем технологии, обучая их создателей

Сайт ВКонтакте Telegram

 133 164
Карма Общий рейтинг

artem @badcasedaily1

Motion – Peter Sandberg

Подписаться



Комментарии 4



-  redArmadillo
24 авг 2023 в 10:03

общее количество документов (3) деленное на количество документов, в которых встречается это слово (2)

А разве это слово встречается не во всех трех документах? Машинное, машинного, машинным...

↑ ↓ Ответить
-  gansior_al
20 июл 2024 в 08:15

Сложно назвать word2vec более современным методом. Создан в 2013 году.

↑ ↓ Ответить



Ты точно знаешь себя?
Проверь свой IT-профиль

начал доходить до мысли, что веса токенов зависят от длины документа в корпусе. Благодаря этой статье получил подтверждение гипотезе - спасибо!

↑ ↓

Ответить

↑ ↓

Ответить

Зарегистрируйтесь на Хабре, чтобы оставить комментарий

Публикации

ЛУЧШИЕ ЗА СУТКИ ПОХОЖИЕ

stas_dubich

17 часов назад

IT-2025: Реквием по здравому смыслу

← Простой

🕒 12 мин

👁 42K

Мнение

💎 +143

🔖 116

💬 119

Loxmatiyamamont

21 час назад

История поиска бага в ядре Linux длиной в год, или неожиданные нули из XFS'a

🔴 Сложный

🕒 11 мин

👁 9.9K

💎 +46

🔖 19

💬 6

ultra_vds

23 часа назад

FTP-сервер на Linux: настройка для домашнего проекта

🔵 Средний

🕒 11 мин

👁 9.3K

Тutorial

💎 +38

🔖 51

💬 9

wmlab

8 часов назад

Путь Солнца Золотого

🟢 Простой

🕒 21 мин

👁 7.6K

💎 +31

🔖 16

💬 12

Lunathecat

8 часов назад

Ты точно знаешь себя?

🟢 Простой

🕒 9 мин

👁 10K

Тutorial



Ты точно знаешь себя?
Проверь свой IT-профиль

+30

9

0



PCB-network

20 часов назад

20 лет в индустрии: история команды SimPCB

9 мин

8К

Интервью

+28

7

12



KarinaKosheva

23 часа назад

Теория и практика интеграции СХД с OpenStack

Средний

13 мин

7.1К

Тutorial

+27

11

2



proDream

22 часа назад

GitLab: Основы написания Pipeline 1/3

Простой

15 мин

9.4К

Тutorial

+24

152

6



cognitronn

20 часов назад

Почему индустрия ИИ начинает трещать по швам?

4 мин

8.9К

Мнение

Перевод

+19

14

51



Xcom-shop

23 часа назад

Как раньше выбирали компьютеры: ностальгический чек-лист из 2000-х

9 мин

21К

+19

6

33

Курс на оптимизацию: как изменились IT-работодатели за 2025 год

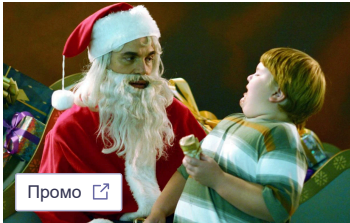
Турбо

Показать еще



Ты точно знаешь себя?

Проверь свой IT-профиль



Дедушка Мороз много скидок нам принёс — забирайте свои



ИИ-ассистент прокомментирует ваш код в статьях на Хабре



Пустынный город или соларпанк? Выбор за вами!

ВАКАНСИИ

Системный аналитик
от 200 000 до 300 000 Р · Zampa · Можно удаленно

Специалист по API и автоматизации данных (Google Apps Script / Python)
до 100 000 Р · Агентство ШОЛЬЧЕВ · Можно удаленно

Аналитик с опытом работы в букмекере
от 1 000 до 150 000 Р · Gravity Group · Можно удаленно

Системный аналитик (Мои Сервисы)
от 300 000 Р · Сбер · Москва

Backend Engineer
от 4 000 до 8 000 \$ · Yolk · Можно удаленно

Больше вакансий на Хабр Карьере

Ваш аккаунт

Войти
Регистрация

Разделы

Статьи
Новости
Хабы
Компании
Авторы
Песочница

Информация

Устройство сайта
Для авторов
Для компаний
Документы
Соглашение
Конфиденциальность

Услуги

Корпоративный блог
Медийная реклама
Нативные проекты
Образовательные программы
Стартапам



Настройка языка

Техническая поддержка

© 2006–2025, Habr



Ты точно знаешь себя?
Проверь свой IT-профиль

ИНФОРМАЦИЯ

Сайт	otus.ru
Дата регистрации	22 марта 2017
Дата основания	1 апреля 2017
Численность	101–200 человек
Местоположение	Россия
Представитель	OTUS

ВИДЖЕТ



Ты точно знаешь себя?
Проверь свой IT-профиль



ВКОНТАКТЕ

 **OTUS. Онлайн-образование**

28 473 подписчика

[Подписаться на новости](#)

БЛОГ НА ХАБРЕ

- 2 часа назад

Настройка .NET Aspire с AWS и LocalStack

 2.3K

 0
- 15 часов назад

Варианты взаимодействия AI-агентов

 9.3K

 2
- 19 часов назад

Создаем простую систему RAG на Python

 6.6K

 0

 **Ты точно знаешь себя?**
Проверь свой IT-профиль

вчера в 09:04

Логи, топики и события: events в Solidity

3.6K 0

ИСТОРИИ

Как меняется работа в data

Код без границ: кто получил гранты?

Год облака в подарок: 2 VM, БД, 100 ГБ

IT-анатомия идеального ремонта

«Зелёная» подборка

Выбирайте курсы по отзывам

На мороз!

Ты точно знаешь себя?

Проверь свой IT-профиль